* **Binary Search Tree** :→ Important Operations :→

(i) Insert Operation → Average O(Log n)

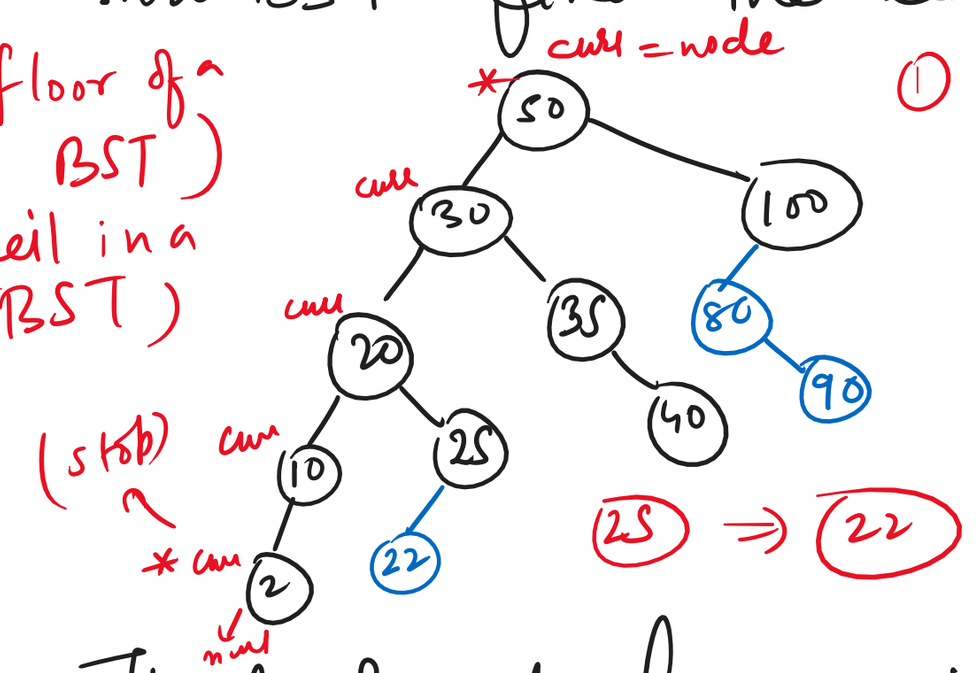
(ii) Search Operation → Average O(log n)

*** (iii) Delete Operation → Important for Tech Interviews

(Left < Node < Right)

In a BST find the least value node of any subtree

(floor of a BST)
(ceil in a BST)



(1)

```
findMin (TreeNode node) {
    Node curr = node;
    while (curr != null && curr.left != null)
    {
        curr = curr.left;
    }
    return curr;
}
```

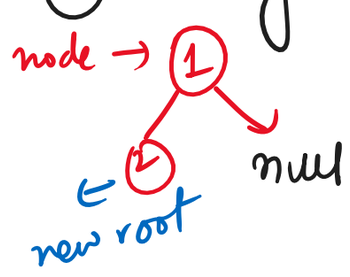
25 ⇒ 22

* The least node from a given node in a subtree or tree in a BST will be to the extreme left.

The two cases of deleting the root node of a BST

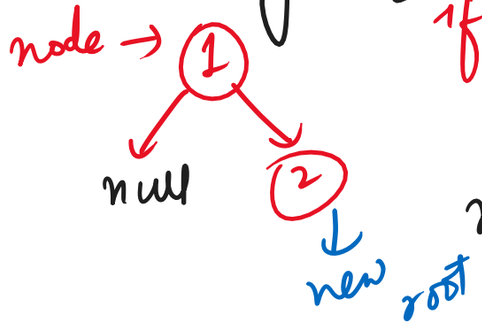
(1) node with one child

(a) only left child




```
if (root.right == null)
    Tree temp = r.l
    root = null;
    return temp;
```

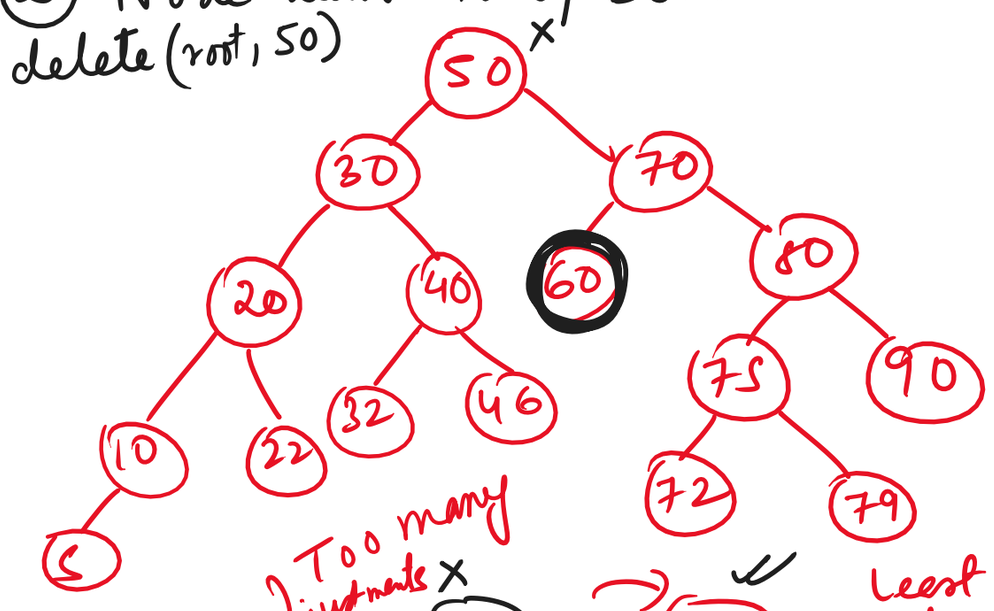
(b) only right child




```
if (root.left == null)
    Tree temp = r.r
    root = null;
    return temp;
```

(2) Node with two/both children

delete (root, 50)

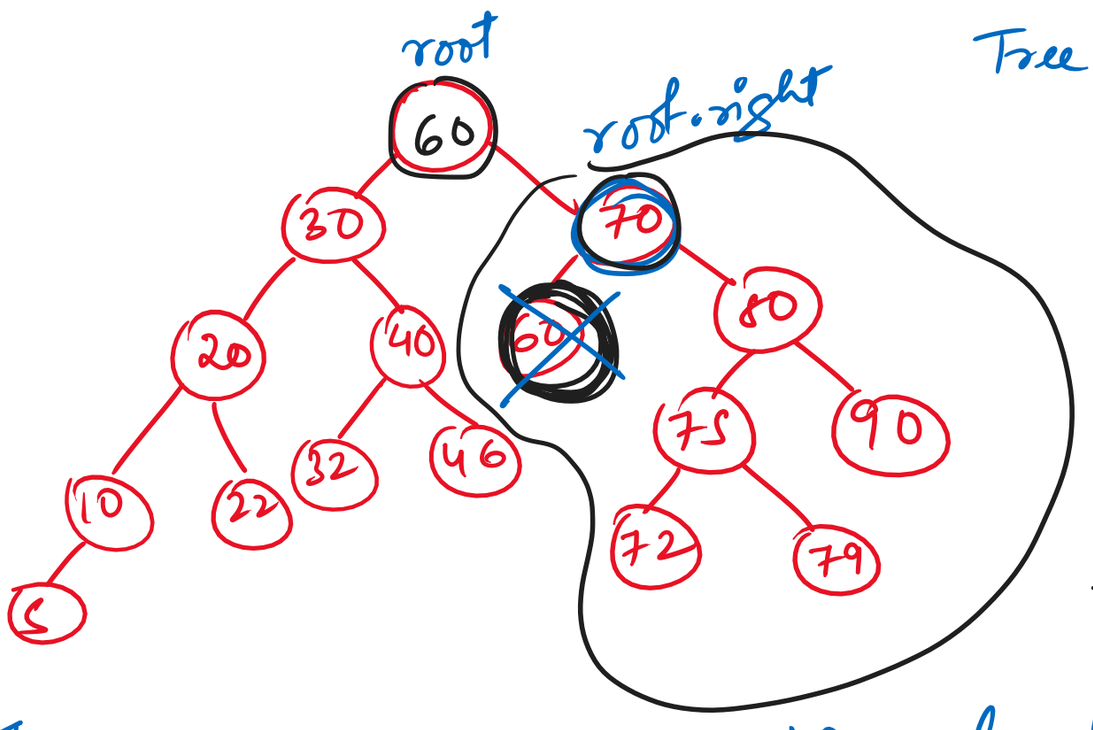


In Order: ... 46 50 60

Too many adjustments ×

Least value of right subtree

(In Order Successor)



Tree temp = findMin (root.right) → 60

temp = 60

root.data = temp.data;

delete (root.right, temp.data)

The inorder success is the least value of the right subtree. When root has two children we replace it by the in-order successor.